

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	PRITHAM G	Reg. No.:	192572014
Section / Batch:	CSE	Date:	20/07/2026

Code of Conduct

I PRITHAM G (192572014) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: PRITHAM G

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:			100 Marks

38	192524251	VAISHNAV M	1. Two algorithms solve the same problem. Algorithm A uses recursion, while Algorithm B uses an explicit stack. Compare both approaches in terms of memory usage, execution flow, and risk of stack overflow. 2. A stack has a capacity of 5 elements. Explain what happens if the following operations are executed: Push(A), Push(B), Push(C), Push(D), Push(E), Push(F). Identify the error and suggest two possible solutions.
39	192525187	VEERA SANTHOSH A	1. Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately. 2. A queue implemented using an array report "Queue Full" even though some positions are empty. ➤ Analyze why this occurs. ➤ Explain how a circular queue solves this problem.
40	192511226	VINIL KUMAR S	1. Consider the singly linked list: $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50$. Delete the node containing 30. Explain how the links change after deletion. 2. Given two sorted linked lists: L1: $1 \rightarrow 3 \rightarrow 5 \rightarrow 7$ L2: $2 \rightarrow 4 \rightarrow 6 \rightarrow 8$

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge

Signature: _____

Date: _____

Course Coordinator

Signature: _____

Date: _____

Program Director

Signature: _____

Date: _____

1) Insert an element at the 3rd position in a doubly linked list having 6 elements, and delete the element at the 2nd position. Implement and analyze it.

⇒ Initial List

Assume the 6 elements are

$10 \rightleftarrows 20 \rightleftarrows 30 \rightleftarrows 40 \rightleftarrows 50 \rightleftarrows 60$

Each node contains:-

Prev Data Next

Step 1: Insert 25 at 3rd position

Before insertion:-

$10 \rightleftarrows 20 \rightleftarrows 30 \rightleftarrows 40 \rightleftarrows 50 \rightleftarrows 60$

After inserting 25 at position 3:

$10 \rightleftarrows 20 \rightleftarrows 25 \rightleftarrows 30 \rightleftarrows 40 \rightleftarrows 50 \rightleftarrows 60$

The links are updated as:

$20 \cdot \text{next} = 25$

$25 \cdot \text{prev} = 20$

$25 \cdot \text{next} = 30$

$30 \cdot \text{prev} = 25$

Step 2: Delete element at 2nd position

The 2nd element is 20:

Before deletion:

$10 \Rightarrow 20 \Rightarrow 25 \Rightarrow 30 \Rightarrow 40 \Rightarrow 50 \Rightarrow 60$

After deletion:

$10 \Rightarrow 25 \Rightarrow 30 \Rightarrow 40 \Rightarrow 50 \Rightarrow 60$

Updated links:

$10 \rightarrow \text{next} = 25$

$25 \rightarrow \text{prev} = 10$

Final list

$\text{Null} \leftarrow 10 \Rightarrow 25 \Rightarrow 30 \Rightarrow 40 \Rightarrow 50 \Rightarrow 60 \Rightarrow \text{NULL}$

Implementation

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node { int d; struct Node *p, *n; };
```

```
int main() {
```

```
    struct Node *h = NULL, *t, *x;
```

```
    int a[] = {10, 20, 30, 40, 50, 60};
```

```
    for (int i=0; i<6; i++) {
```

```
        x = malloc(sizeof(*x)); x->d = a[i];
```

```
        x->n = NULL;
```

```
        if (!h) h = x;
```

```
        else { t->n = x; x->p = t; }
```

```
        t = x;
```

```
    }
```

$t = h \rightarrow n \rightarrow n;$

$x = \text{malloc}(\text{sizeof}(*x)); x \rightarrow d = 25;$

$x \rightarrow p = t \rightarrow p; x \rightarrow n = t;$

$t \rightarrow p \rightarrow n = x; t \rightarrow p = x;$

$t = h \rightarrow n;$

$t \rightarrow p \rightarrow n = t \rightarrow n; t \rightarrow n \rightarrow p = t \rightarrow p;$

~~$\text{free}(t);$~~

$\text{for}(t = h; t; t = t \rightarrow n) \text{print}("%d", t \rightarrow d);$

3

- 2 Perform the following operations using stack. Assume the size of the stack is 5 and have a value of 22, 55, 33, 66, 88 in the stack from 0 position to size-1. Now perform the following operations:
- 1) Invert the element in the stack, 2) $\text{pop}()$, 3) $\text{pop}()$, 4) $\text{Push}(90)$, 5) $\text{Push}(36)$, 6) $\text{Push}(11)$, 7) $\text{Push}(88)$, 8) $\text{Pop}()$, 9) $\text{Pop}()$.
- Draw the diagram of the stack and illustrate the above operations and identify where the top is?

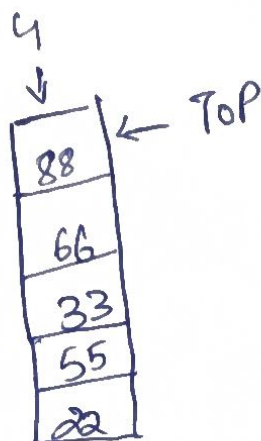
⇒ Given:

→ Stack size = 5

→ Initial element: 22, 55, 33, 66, 88

→ Positions: 0 to 4

→ Initially TOP = 4



1) Invert the element

The elements are reversed

Before:

22 55 33 66 88

After inversion:

88 66 33 55 22

So: TOP = 4

2) Pop()

22 is removed because it is at the TOP after inversion

88 66 33 55 ← TOP

TOP = 3

3) Pop()

55 is removed.

TOP = 2

4) Push (90)

90 is added to the top

88 66 33 90 $\leftarrow$ TOP

TOP = 3

5) Push (36)

36 is added

88 66 33 90 36 $\rightarrow$ TOP
TOP = 4

6) Push (11)

The stack is already full

88 66 33 90 36 $\leftarrow$ TOP

Therefore:

Stack overflow:

11 is not inserted

7) Push (88)

The stack is still full

Therefore:

Stack overflow, 88 is not inserted.

8) Pop()

36 is removed.

88 66 33 90 $\leftarrow$ TOP

TOP = 3

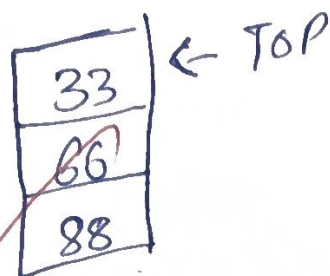
9.1) POP()

90 is removed

88 66 33 ← TOP

TOP = 2

Final stack



Final result

Final stack = 88, 66, 33

TOP = 2

Operation Summary

No	Operation	Result	TOP
		88, 66, 33, 55, 22	4
1	Invert	88, 66, 33, 56	3
2	POP()	88, 66, 33	2
3	POP()		
4	Push(90)	88, 66, 33, 90	3
5	Push(36)	88, 66, 33, 90, 36	4
6	Push(11)	overflow	4
7	Push(88)	overflow	4
8	POP()	88, 66, 33, 90	3
9	POP()	88, 66, 33	2